Software Requirements Specification (SRS)

KVD Estates

---

1. Introduction

1.1 Purpose

This document provides a detailed description of the requirements for the KVD Estates application. The purpose of this system is to enable users to search, view, manage, and list real estate properties efficiently, providing streamlined property management and facilitating seamless interactions between buyers, renters, and admins.

1.2 Scope

KVD Estates is a comprehensive web-based real estate management platform designed to streamline the property discovery and management process. The application allows users to effortlessly search, and filter available properties based on their specific criteria, save and shortlist their favorite listings for future reference, and view detailed property profiles complete with high-quality images.

Beyond property discovery, the system provides robust tools for property management and user interaction. It enables admins and administrators to manage property listings, client portfolios, and inquiries, as well as upload and publish new property data efficiently. Furthermore, KVD Estates supports direct and seamless communication between buyers, renters, and real estate admins, ensuring a smooth end-to-end experience for all parties involved.

1.3 Definitions

1.3.1 Listing: A detailed profile of a real estate property available for sale or rent, including specifications, images, and pricing.
1.3.2 Portfolio: A collection of properties actively managed by a real estate admin or administrator.
1.3.3 Shortlist: A curated selection of properties saved by a user for future consideration and quick access.

2. Overall Description

2.1 Product Perspective

KVD Estates is a comprehensive web-based application that seamlessly integrates a robust property database management system, an advanced search and filtering engine for discovering live listings, and a secure data management system tailored for handling role-based user access (Seekers, Owners, and Admins) and property records.

2.2 Product Functions

The KVD Estates system enables Seekers to search, filter by address and maximum budget, and view detailed real estate listings from a centralized database, culminating in a

simulated secure booking pipeline. It provides dedicated property management tools for Owners to securely create listings using interactive map-based reverse geocoding and batch media uploads. Additionally, the platform supports a strict administrative moderation workflow to transition newly submitted properties from a pending state to a live, publicly viewable status.

2.3 User Classes

The KVD Estates platform strictly enforces access control via three defined user roles:

- **Seekers:** Prospective renters who utilize the system to search the catalog of live properties, view high-resolution media carousels, calculate financial payloads, and execute property bookings.
- **Owners:** Property listers who require access to create new property entries, designate locations via map pins, and upload property image galleries.
- **Admins:** Highly privileged users with the sole authority to moderate the platform, approve pending property listings to make them live, and securely view sensitive Seeker and booking records.

2.4 Operating Environment

The KVD Estates application is designed to operate seamlessly across all major web browsers on both desktop and mobile operating systems, including Windows, macOS, Android, and iOS. Reliable internet connectivity is required for users to access real-time property listings, process secure mock payments, upload new property media, and manage property approvals effectively

2.5 Constraints

The development and operation of the KVD Estates platform are subject to several constraints. The system's location, mapping, and reverse geocoding features are strictly dependent on the continuous availability and integration of OpenStreetMap and Leaflet JS. Additionally, the platform must effectively manage local server-side storage (via SQLite3 and local media roots) to accommodate the volume of high-resolution images uploaded through the custom Django MultipleFileField. Finally, the system must strictly adhere to data privacy standards when Admins access confidential Seeker booking details.

3.1 Functional Requirements

3.1.1 User Roles and Access Control

- 3.1.1.1 Role-Based Authentication: The system shall utilize the Django User model, customized with strict boolean flags to categorize users into three distinct roles: Admin, Owner, and Seeker.
- 3.1.1.2 Dynamic Routing: Upon successful authentication, the system's Main Dashboard shall dynamically route users to exclusive visual interfaces and toolsets based on their assigned role flag.

3.1.2 Property Listing Management (Owner & Admin)

- 3.1.2.1 Owner Property Creation: Property Owners shall be able to create new

property listings. During creation, the system shall provide an interactive OpenStreetMap and Leaflet JS interface, allowing Owners to drop a pin to auto-complete the property address via Reverse Geocoding.

- 3.1.2.2 Batch Media Uploads: The system shall implement a custom Django MultipleFileField, permitting Owners to select and upload multiple property gallery photos simultaneously via a single input array.
- 3.1.2.3 Listing State Management & Moderation: Newly submitted property listings shall mechanically default to a Pending state. Only users with the Admin role shall possess the authority to approve a listing, transitioning its state from Pending to Live. Admins shall also have access to view all properties regardless of their current state.

3.1.3 Search and Booking Operations (Seeker)

- 3.1.3.1 Catalog Filtering: Seekers shall be presented with a catalog containing exclusively Live properties. The system shall allow Seekers to filter this catalog using address matching and maximum budget constraints.
- 3.1.3.2 Detailed Property View: The system shall provide a full-screen details view for properties, incorporating Bootstrap Image Carousels for media viewing and interactive neighborhood maps.
- 3.1.3.3 Dynamic Financial Calculator: The system shall feature an embedded JavaScript calculator to instantly display the Total Payload for a booking. This payload shall be calculated as (Property Rent * Duration) + 1 Month Deposit. Both frontend and backend validations shall strictly block negative numerical inputs.
- 3.1.3.4 Checkout and Payment Simulation: Initiating a booking shall set the transaction to a Pending state and redirect the Seeker to a simulated Razorpay Secure Checkout Overlay. Successful resolution of this mock payment shall finalize the transaction and automatically mutate the target property's status to Booked.

3.2 Non-Functional Requirements

3.2.1 Availability The system must maintain an uptime of 99% during standard business hours to ensure reliable access for all users of the KVD Estates platform.
3.2.2 Performance To provide a seamless browsing experience, any search query executed within the system must process and return results in under one second.
3.2.3 Scalability The application architecture must support high traffic, successfully accommodating a concurrency of up to 10,000 simultaneous users. Additionally, the underlying database must be structured to be vertically scalable to handle future data and traffic growth.
3.2.4 Security All user passwords must be securely encrypted utilizing the SHA-256 cryptographic hash algorithm to protect sensitive authentication credentials from unauthorized access.

3.3 System Constraints

3.3.1 Technology Stack The development of the project is strictly constrained to a predefined technology stack. The user interface must be constructed using HTML, JavaScript, and CSS, while the backend architecture must be powered by the Django framework paired with a SQL lite relational database.
3.3.2 Schedule The complete software solution, including all development and testing phases, must be finalized and delivered within a strict timeframe of 10 weeks.

4. System Features Summary

- Search and filter real estate listings
- Portfolio creation and management
- Detailed property profile viewing
- Secure listing creation and publishing

5. Future Enhancements

• AI-based property recommendations based on user search behavior • Native mobile application development for iOS and Android • Integration of 3D virtual tours and Augmented Reality (AR) viewing • Advanced financial tools, including integrated mortgage and investment calculators

6. System Architecture

6.1 High-Level Architecture

The KVD Estates system follows a modular architecture consisting of three primary layers. The **User Interface (UI) Layer** handles all client interactions, displaying property listings, search filters, admin portfolios, and interactive maps. The **Application Logic Layer** manages the core business operations and coordinates between specific functional modules. These include a Search Module for handling property queries, a Data Fetch Module for retrieving listing metadata, a Media Manager for processing image and document uploads, a Gallery/Map Engine for viewing high-resolution images and locations, a Portfolio Manager for handling user shortlists and admin listings, and a Storage Manager for local caching. Finally, the **Data Layer** relies on a robust database to securely store property metadata, user profiles, transactional history, and saved searches.

6.2 Data Flow

- Search Flow: Seeker → UI → Search Module (Address & Max Budget Filters) → Database Fetch (Live Properties Only) → Results → UI

- Listing Creation Flow: Owner → UI → Map Interface (Reverse Geocoding) & Media Array (MultipleFileField) → Database Update (Status: Pending) & Local Storage → UI

- Admin Approval Flow: Admin → UI → Moderation Dashboard → Approve Action → Database Update (Mutate Pending to Live) → UI

- Property Viewing Flow: Seeker → UI → Request Listing Details → Database Retrieval → UI Display (Bootstrap Carousels & Interactive Leaflet Map)

- Booking & Checkout Flow: Seeker → UI → Input Duration → JS Calculator (Rent * Duration + Deposit) → Simulated Razorpay Overlay → Payment Resolution → Database Update (Mutate Status to Booked) → UI

6.3 Technology Stack

- **Frontend:** HTML5, Vanilla CSS, Bootstrap 5
- **Backend:** Django
- **Database:** SQL lite(for user data, property listings, and metadata)
- **Authentication:** Django (for secure buyer, renter, and admin logins)
- **Mapping & Media:** Leaflet JS (for property locations) and Pillow (for image optimization and delivery)

7 Conclusion

KVD Estates aims to provide seamless property management experience by combining intuitive property discovery, secure listing management, and robust portfolio organization into a single comprehensive platform, with a strong focus on usability and performance.